Arrays

LC 217 → Contains Duplicate.

LC 242 → Valid Anagram.

LC 1 → Two Sum.

0	1	2	3
3,	4,	5,	6

9

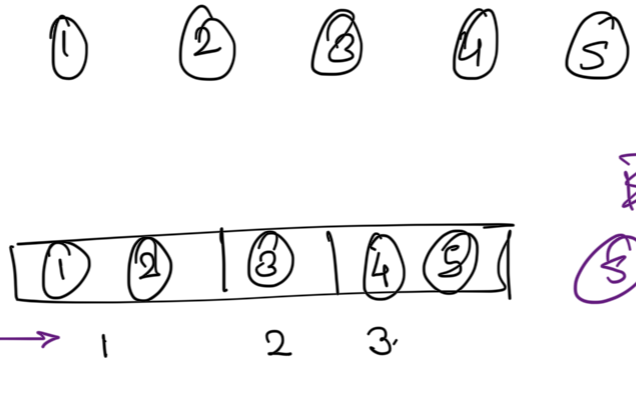
~~6~~ ~~5~~ 4 → 2, 1.

0	3
1	4

key = diff	value = index
3	0
4	1

LC-49 → Group Anagrams.

[eat, tea, tan, ate, nat, bat]



But what if 5 & 1 are anagrams

Our res_ita cant come back → ∴ this approach can only work if we sort the strs array

which cant be done

Maintain a map.

Key	value
sorted_word	<vector> of strings

LC-347 Top-k Frequent Elements.

Map

$k = 3$

Key	Value
1	23
2	2
3	51
4	36
5	15
6	11
7	69

n log n

sort → Sorting

<value, index>

23
2
51
36
15

size = n →

map

k	v.
1	23

if a[in] >

n.

result

result_min.

O(n) → traversal on freq_map.

Map

Key	value
1	23
2	3
3	51

result

1	2	3	4	5

min_key → 2

4	36
5	15
6	11
7	69

min_index → 1
min_freq → ~~INT_MAX~~ ~~23~~ 3
space = 0
cnt = 5.

→ If space > 0

→ Check pair.second < min_freq.

→ Yes

→ min_k = pair.first
→ min_index = cnt **OR** (k - space)
→ min_freq = pair.second.

→ result [cnt] = pair.first
→ space --;
→ cnt ++;

No Space → Hence Replace.

→ Here we cant go back. And remove smallest.

Eg: freq → 1 2 3 4 5 6, 100, 2500, 100.

1, 2, 3, 4, 5 → for 100
→ we remove 1

& update min_{___}
for 100.

∴ 100, 2, 3, 4, 5

→ for 2500 → we would replace 100 BUT
we should actually replace 2,
which we have no data about.

⇓

HEAP. (using priority queue).

1. Freq Map
2. Push each {key, value} onto a Min-heap.
   → At the end we would have
   "k" elements in heap.
3. Extract keys into result vector

PQ compares pairs by FIRST Element
of Pair.

⇓

∴ first should be freq.

LCXXX — Encode & Decode Strings

→ [add length + delimiter]

This will be our delimiter.

hello → 5# hello

LC238 — Product of Array Except Self.

[1, 2, 3, 4]

Prefix product → [1, 2, 6, 24]

Suffix product → [24, 24, 12, 4]

Req. answer → [24, 12, 8, 6].

i=0. In result.

prefix [i-1] * suffix [i+1].

Eg 2: [-1, 0, 1, 2, 3]

prefix [-1, 0, 0, 0, ~~0~~]

suffix [~~0~~, 0, 6, 6, 3]

result. [0, -6, 0, 0, 0]

Need to multiply by 1 as added directly.

never visited.